

```
# Turn switch on, and raise the level 3 times
oDimmer.turnOn()
oDimmer.raiseLevel()
oDimmer.raiseLevel()
oDimmer.raiseLevel()
oDimmer.show()
```

Listing 2-6: DimmerSwitch class with test code

When we run this code, the resulting output is:

```
Switch is on? True
Brightness is: 5
Switch is on? False
Brightness is: 3
Switch is on? True
Brightness is: 6
```

The main code creates the `oDimmer` object, then makes calls to the various methods. Each time we call the `show()` method, the on/off state and the brightness level are printed. The key thing to remember here is that `oDimmer` represents an object. It allows access to all methods in the class from which it was instantiated (the `DimmerSwitch` class), *and* it has a set of all instance variables defined in the class (`self.switchIsOn` and `self.brightness`). Again, instance variables maintain their values between calls to methods of an object, so the `self.brightness` instance variable is incremented by 1 for each call to `oDimmer.raiseLevel()`.

Representing a More Complicated Physical Object as a Class

Let's consider a more complicated physical object: a television. With this more complicated example, we'll take a closer look at how arguments work in classes.

A television requires much more data than a light switch to represent its state, and it has more behaviors. To create a `TV` class, we must consider how a user would typically use a TV and what the TV would have to remember.